Unit 2

1) Define a function that computes the length of a given list or string.

```
def find_length(data):
    count = 0
    for i in data:
        count += 1
    return count

# List
my_list = [10, 20, 30, 40, 50]
print("Length of List =", find_length(my_list))

# String
my_string = "Python"
print("Length of String =", find_length(my_string))
```

o/p
Length of List = 5
Length of String = 6

2) Print a specified list after removing the 0th, 2nd, 3rd and 5th elements.

```
color = ['Red', 'Green', 'White', 'Black', 'Pink', 'Yellow']

result = [x for i, x in enumerate(color) if i not in (0, 2, 3, 5)]

print("Original List:", color)
print("New List:", result)
```

o/p
Original List: ['Red', 'Green', 'White', 'Black', 'Pink', 'Yellow']
New List: ['Green', 'Pink']

3) Sort a dictionary by value (Ascending and Descending)

```
d = {'A': 50, 'B': 10, 'C': 30, 'D': 20}

ascending = dict(sorted(d.items(), key=lambda item: item[1]))
descending = dict(sorted(d.items(), key=lambda item: item[1], reverse=True))

print("Ascending:", ascending)
print("Descending:", descending)
```

o/p
Ascending: {'B': 10, 'D': 20, 'C': 30, 'A': 50}
Descending: {'A': 50, 'C': 30, 'D': 20, 'B': 10}

4) Sum all the items in a dictionary

```
d = {'A': 10, 'B': 20, 'C': 30}

total = sum(d.values())

print("Dictionary:", d)
print("Sum =", total)
```

o/p
Dictionary: {'A': 10, 'B': 20, 'C': 30}

Sum = 60

5) Searching and Sorting techniques

```
# ========== Searching ==========

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

def binary_search(arr, key):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# ========== Sorting ==========

def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
```

```
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
```

```
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

# ========== Testing ==========
data = [64, 34, 25, 12, 22, 11, 90]
print("Original list:", data)

print("\n--- Searching ---")
print("Linear Search of 22:", linear_search(data, 22))
sorted_data = sorted(data)
print("Binary Search of 22 (on sorted list):",
binary_search(sorted_data, 22))

print("\n--- Sorting ---")
print("Selection Sort:", selection_sort(data.copy()))
print("Bubble Sort   :", bubble_sort(data.copy()))
print("Insertion Sort:", insertion_sort(data.copy()))
print("Merge Sort    :", merge_sort(data.copy()))
print("Quick Sort    :", quick_sort(data.copy()))
```

o/p

```
Original list: [64, 34, 25, 12, 22, 11, 90]

--- Searching ---
Linear Search of 22: 4
Binary Search of 22 (on sorted list): 3
```

```
--- Sorting ---
Selection Sort: [11, 12, 22, 25, 34, 64, 90]
Bubble Sort   : [11, 12, 22, 25, 34, 64, 90]
Insertion Sort: [11, 12, 22, 25, 34, 64, 90]
Merge Sort    : [11, 12, 22, 25, 34, 64, 90]

Quick Sort    : [11, 12, 22, 25, 34, 64, 90]
```

6) Encapsulation concept

```
class Student:
    def __init__(self, name, marks):
        self.__name = name      # private attribute
        self.__marks = marks    # private attribute

    # Getter methods
    def get_name(self):
        return self.__name

    def get_marks(self):
        return self.__marks

    # Setter methods
    def set_name(self, name):
        self.__name = name

    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks! Must be between 0 and 100.")

# Testing
```

```
s = Student("Rahul", 85)
print("Name:", s.get_name())
print("Marks:", s.get_marks())

s.set_marks(92)
print("Updated Marks:", s.get_marks())

s.set_marks(150)   # Invalid
```

o/p

```
Name: Rahul
Marks: 85
Updated Marks: 92

Invalid marks! Must be between 0 and 100.
```

Unit 3

7) Different plotting using PyLab (Line, Bar, Pie, Histogram, Scatter)

```
from pylab import *

# 1. Line Plot
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]
figure()
plot(x, y, marker='o')
title("Line Plot")
xlabel("X-axis")
ylabel("Y-axis")
grid(True)
show()

# 2. Bar Chart
categories = ['A', 'B', 'C', 'D']
values = [25, 40, 30, 55]
figure()
bar(categories, values, color='skyblue')
title("Bar Chart")
xlabel("Categories")
ylabel("Values")
show()

# 3. Pie Chart
labels = ['Python', 'Java', 'C++', 'JavaScript']
sizes = [35, 25, 20, 20]
figure()
pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90)
title("Pie Chart")
axis('equal')
show()
```

```
# 4. Histogram
data = [22, 25, 30, 35, 40, 42, 45, 50, 55, 60, 22, 28, 33]
figure()
hist(data, bins=5, color='green', edgecolor='black')
title("Histogram")
xlabel("Value")
ylabel("Frequency")
show()

# 5. Scatter Plot
x = [5, 7, 8, 7, 2, 17, 2, 9, 4, 11]
y = [99, 86, 87, 88, 111, 86, 103, 87, 94, 78]
figure()
scatter(x, y, color='red')
title("Scatter Plot")
xlabel("X-axis")
ylabel("Y-a


xis")
show()
```

8) Plotting a curve

```
from pylab import *
import numpy as np

x = np.linspace(-2 * np.pi, 2 * np.pi, 400)
y = np.sin(x)

figure()
plot(x, y, color='blue', linewidth=2)
```

```
title("Sine Curve")
xlabel("x")
ylabel("sin(x)")
grid(True)
axhline(0, color='black', linewidth=0.5)
axvline(0, color='black', linewidth=0.5)

show()
```

9) 0/1 Knapsack algorithm

```
def knapsack(W, wt, val, n):
    # Create DP table
    K = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                K[i][w] = 0
            elif wt[i - 1] <= w:
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]],
                              K[i - 1][w])
            else:
                K[i][w] = K[i - 1][w]
    return K[n][W]

# Example
val = [60, 100, 120]
wt = [10, 20, 30]
W = 50
n = len(val)

print("Maximum value that can be obtained:", knapsack(W, wt,
val, n))
```

o/p
Maximum value that can be obtained: 220

10) Divide and Conquer algorithm (Example: Maximum of a list)

```
def find_max(arr, low, high):
    # Base case: only one element
    if low == high:
        return arr[low]

    # Divide
    mid = (low + high) // 2

    # Conquer
    left_max = find_max(arr, low, mid)
    right_max = find_max(arr, mid + 1, high)

    # Combine
    return max(left_max, right_max)

# Testing
numbers = [12, 45, 67, 23, 89, 34, 11]
print("List:", numbers)
print("Maximum element (using Divide & Conquer):",
find_max(numbers, 0, len(numbers) - 1))
```

o/p
List: [12, 45, 67, 23, 89, 34, 11]
Maximum element (using Divide & Conquer): 89